

Code Review

Software Engineering for Scientists

Fill this in after reading the request and the code. Leave line comments on the code itself, and use this summary to tie them together. Start every comment with one of the labels below.

Label	Meaning	Author must act?
Required	A bug, a wrong result, or a missing test for core behavior. Backed by a fact or a run.	Yes
Suggestion	A change you think improves the code. Explain why.	Author decides
Question	Something you do not understand yet.	Reply needed
Preference	Style you like that is not a project rule. Use rarely.	No
Praise	Something done well, or something you learned.	No

Overview

Request reviewed: Building the peptide mass database

Reviewer: a classmate

Files reviewed: `src/gen_spectra.py`, `src/preprocessing/merge_search.py` at commit 3b8557e

Summary

What works well

The math in `max_mass` is right for charges 1 and 2. I checked it against `b_ions` and `y_ions` in the same file and the full-length values match exactly. `get_data` is short and easy to follow.

Most important change

`max_mass` returns a wrong answer instead of an error when it gets an ion or charge it does not support. A guard clause fixes this.

Answers to the author's questions

1. Charge handling is correct for 1 and 2. The doubly charged base already includes the second proton, so dividing by 2 gives the true m/z. Charge 3 is not handled (see Required 1).
2. `end` is exclusive. The caller sets `end = start + size`, so `prot[start:end]` is the k-mer. That is 0-based half-open, the same convention as Python slices and BED files. I would update your notes and the docstring to say so.
3. Yes, both functions are pure, which makes them easy to test. I suggest four tests under Testability.

Correctness (safe from bugs)

- ☒ I ran the code, or the example in the request, and got the stated result
- ☐ Inputs outside the stated range fail loudly instead of returning a wrong answer
- ☒ Units, coordinates and conventions match the documentation (once "closed" becomes "half-open")
- ☐ Edge cases (empty input, length 1, unexpected values) behave sensibly

Comments

Required 1. [gen_spectra.py L49 to 60](#). Unsupported inputs return a plausible but wrong number. I ran these two calls.

```
max_mass('DPQVAQLELGG', ion='x', charge=1) # 1108.563 (treated as a b ion)
max_mass('DPQVAQLELGG', ion='b', charge=3) # 369.857 (doubly charged base / 3)
```

Neither raises an error, so a typo upstream would quietly corrupt the database. A guard at the top of the function makes this fail fast.

```
if ion not in ('b', 'y'):
    raise ValueError(f"ion must be 'b' or 'y', got {ion!r}")
if charge not in (1, 2):
    raise ValueError(f"charge must be 1 or 2, got {charge!r}")
```

Question 1. [constants.py L84 to 85](#). `SINGLY_CHARGED_B_BASE` is a hydrogen atom minus an electron, which is a proton. `PROTON_MASS` is defined separately on line 79. They agree to about $1e-7$ Da. Was the separate definition intentional? Building the bases from `PROTON_MASS` would keep the two from drifting apart.

Modularity

- ☒ Each function has one clear purpose
- ☒ Functions in a file are related
- ☐ No duplicated logic (Don't Repeat Yourself)
- ☒ Modules are not tightly coupled

Comments

Suggestion 1. The b and y branches in `max_mass` repeat the same sum, and `mz = total / charge` appears twice. A small lookup table removes the duplication and makes the guard clause above natural.

```
ION_BASE = {
    ('b', 1): SINGLY_CHARGED_B_BASE, ('b', 2): DOUBLY_CHARGED_B_BASE,
    ('y', 1): SINGLY_CHARGED_Y_BASE, ('y', 2): DOUBLY_CHARGED_Y_BASE,
}
```

```
def full_ion_mz(sequence: str, ion: str, charge: int) -> float:
    if (ion, charge) not in ION_BASE:
        raise ValueError(f"unsupported ion/charge: {ion!r}, {charge!r}")
    residues = sum(AMINO_ACIDS[aa] for aa in sequence)
    return (ION_BASE[(ion, charge)] + residues) / charge
```

This is a suggestion. The current version is correct for valid input, so keep it if you prefer.

Testability

- ☒ Functions return results instead of printing them
- ☐ Unit tests exist for the functions that need them
- ☐ Tests cover the normal case and the edge cases

Comments

Suggestion 2. Your own `b_ions` and `y_ions` make a good oracle. The last value of each series is the full-length ion, which is what `max_mass` should return.

```
import pytest
from gen_spectra import max_mass, b_ions, y_ions
from preprocessing.merge_search import get_data
```

```
SEQ = 'DPQVAQLELGG'
```

```

@pytest.mark.parametrize('ion, series', [('b', b_ions), ('y', y_ions)])
@pytest.mark.parametrize('charge', [1, 2])
def test_max_mass_matches_ion_series(ion, series, charge):
    assert max_mass(SEQ, ion, charge) == pytest.approx(series(SEQ,
charge)[-1])

def test_max_mass_rejects_bad_ion():
    with pytest.raises(ValueError):
        max_mass(SEQ, 'x', 1)

def test_max_mass_rejects_charge_3():
    with pytest.raises(ValueError):
        max_mass(SEQ, 'b', 3)

def test_get_data_returns_one_row_per_charge():
    rows = get_data(SEQ, 0, len(SEQ), 7, 'y')
    assert [r[4] for r in rows] == [1, 2]
    assert all(r[3] == 1 and r[5] == 7 for r in rows)

```

The two `rejects` tests fail today and pass once Required 1 is in.

Performance

- ☒ No unnecessary loops or repeated work
- ☒ No unnecessary data structures or copies
- ☒ I would write it about the same way given the inputs and outputs

Comments

No comments. `get_data` sums the residues once per charge, but that is cheap next to the database insert.

Documentation

- ☐ A docstring says what each function does, with an example if useful
- ☐ Parameters and return values have type hints
- ☐ Each parameter has a clear description, including units
- ☐ Inline comments explain why, not what, and there is no dead code

Comments

Suggestion 3. Your request already has great descriptions of both functions. Moving them into docstrings would put them where the next reader looks. Please include "0-based, half-open" for `start` and `end`, and "Daltons per unit charge" for the return value.

Suggestion 4. [merge_search.py L47 to 58](#). The trailing `#1108.563316435` and the commented-out `DPQVAQLELGG` check look like debugging notes. That value would make a nice test case, so consider moving it into the tests and deleting the comments.

Legibility and style

- ☒ Consistent spacing around parentheses, operators and commas
- ☒ `snake_case` for functions and variables, `CamelCase` for classes
- ☐ No magic numbers; constants are named

- ☒ Lines fit on the screen (about 80 characters)
- ☐ Names describe what things are or do, and are spelled correctly

Comments

Question 2. Why is it called `max_mass`? I think it is because the full-length ion is the largest in its series, but I needed `b_ions` to figure that out. A name like `full_ion_mz` would say what it returns. Either way, `seqeunce` is a typo for `sequence`.

Suggestion 5. `ion_int = 0 if ion == 'b' else 1` stores a code the database reader must remember. Named constants such as `B_ION = 0` and `Y_ION = 1` in `constants.py` would document it in one place.

Praise. The comments next to each base mass in `constants.py` explain the chemistry, such as which atoms turn the C-terminus into COOH. That made this review possible for someone outside proteomics.

What I learned

I did not know b and y ions start from different base masses. The y base includes a whole water molecule because the y fragment keeps the original C-terminus.